FDSI · GUÍA ALTERNATIVA · 2026-2

Reverse Engineering Challenge Lab

No necesitas publicar ningún servicio. Todo ocurre sobre binarios compilados y objetivos deliberadamente vulnerables/autorizados.

🧩 ELF x86-64 | 🧠 Ghidra | 🔬 strings / readelf / objdump | 🐞 GDB | 🌐 Burp opcional | 📦 GitHub evidence

🚨 Alcance

Los binarios de esta práctica son **crackmes académicos creados para ser analizados**. No descargues ejecutables desconocidos ni hagas ingeniería inversa sobre software para el cual no tengas autorización. Si utilizas Burp, trabaja únicamente con los laboratorios de PortSwigger o aplicaciones locales asignadas por el docente.

🎯 La misión

Nivel 1 · Recon

Descubre cómo valida el programa una contraseña utilizando análisis estático básico. Obtén su FLAG.

Beginner

Nivel 2 · Decompile

La clave ya no aparece en texto plano. Reconstruye el algoritmo de validación con Ghidra y confirma dinámicamente con GDB.

Intermediate

Boss · Stripped

Repite el Nivel 2 usando la versión sin símbolos. Identifica funciones por comportamiento, referencias y flujo.

Challenge

0. Preparación · 10 puntos

Entorno recomendado: Kali Linux o Ubuntu x86-64. Los binarios entregados son ELF Linux x86-64.

```
sudo apt update
sudo apt install -y binutils gdb file
# Ghidra: instalar desde la distribución oficial/aprobada por el docente.

mkdir -p ~/fdsi-reverse/evidence
cd ~/fdsi-reverse
```

- [x] Copié los tres binarios al directorio de trabajo.
- [x] Ejecuté `file` para confirmar formato y arquitectura.
- [x] Calculé SHA-256 y lo registré en mi informe.
- [x] Creé `docs/evidence/reverse/` en mi repositorio GitHub.
- [x] No abrí ni solicité el código fuente del crackme.

1. Baseline forense · 10 puntos

```
file crackme_level1
file crackme_level2
file crackme_level2_stripped

sha256sum crackme_level1 crackme_level2 crackme_level2_stripped

readelf -h crackme_level1
readelf -h crackme_level2
```

Hashes esperados del material original:

Archivo	SHA-256
crackme_level1	`61e980febe84b1003b5a3b641468e915b984f7fdd835be9828af54233f88c68c`
crackme_level2	`8dc5931dfbf74d7371de9ca9ed8cc57bfe0af4521346202dcd1c701dd8b6f4e5`
crackme_level2_stripped	`c8e638741272a87ee3b30fe8878898c1aa977e6a879a1ec0b271034b5bb9aed3`

- [x] Identifiqué arquitectura, formato, linking y presencia/ausencia de símbolos.

- [x] Expliqué por qué un hash ayuda a garantizar que todos analizan el mismo binario.

2. Nivel 1 — “Strings are evidence” · 15 puntos

Primero ejecuta el programa con datos falsos. Después observa sin asumir.

```
chmod +x crackme_level1
./crackme_level1
./crackme_level1 prueba

strings -n 5 crackme_level1 | less
objdump -d -M intel crackme_level1 | less
```

- [x] Identifiqué mensajes relevantes del binario.

- [x] Localicé una cadena que parece relacionada con autenticación/validación.

- [x] Probé mi hipótesis ejecutando el binario.
- [x] Obtuve la FLAG del Nivel 1 sin consultar el código fuente.
- [x] Documenté por qué `strings` puede revelar secretos embebidos.

3. Nivel 2 — Ghidra: reconstruir la validación · 25 puntos

1. Crea un proyecto **Non-Shared** en Ghidra.
2. Importa `crackme_level2`.
3. Acepta el análisis automático.
4. Ubica `main` y sigue la llamada que decide si la licencia es válida.
5. Renombra variables y funciones según lo que deduzcas.
6. Escribe pseudocódigo propio del algoritmo.

```
# Complemento fuera de Ghidra
strings -n 5 crackme_level2
objdump -d -M intel crackme_level2 | less
objdump -s -j .rodata crackme_level2 | less
```

- [x] Localicé la función de validación desde `main`.
- [x] Identifiqué la longitud esperada de la clave.
- [x] Identifiqué la transformación aplicada a cada byte.
- [x] Reconstruí la clave correcta y obtuve la FLAG.
- [x] Guardé captura del decompilador con mis anotaciones/renombres.

4. Confirmación dinámica con GDB · 15 puntos

No se trata de “adivinar”; se trata de demostrar en ejecución que tu hipótesis está bien.

```
gdb ./crackme_level2

# Dentro de GDB:
set disassembly-flavor intel
break validate_key
```

```
run AAAA
disassemble validate_key
info registers
continue
quit
```

Después repite con tu clave candidata y observa cómo cambia el retorno de la función.

- [x] Detuve la ejecución en la rutina de validación.
- [x] Relacioné registros/instrucciones con el pseudocódigo de Ghidra.
- [x] Confirmé dinámicamente una clave fallida y una válida.

5. Boss Level — binario sin símbolos · 10 puntos

Ahora abre `crackme_level2_stripped`. Ya no debes depender de nombres como `validate_key`.

```
file crackme_level2_stripped
nm crackme_level2_stripped
strings -n 5 crackme_level2_stripped
objdump -d -M intel crackme_level2_stripped | less
```

- [x] Expliqué qué información desapareció al hacer strip.
- [x] Encontré de nuevo la lógica de validación usando referencias y flujo de control.

6. Evidencia y entrega GitHub · 10 puntos

```
docs/
└── evidence/
    └── reverse/
        ├── baseline.txt
        ├── level1.md
        ├── level2.md
        ├── gdb.md
        └── screenshots/
```

```
reverse-analysis.md
README.md
```

- [x] `baseline.txt` contiene hashes, `file` y `readelf`.
- [x] `level1.md` explica hipótesis → evidencia → resultado.
- [x] `level2.md` contiene pseudocódigo propio, no código decompilado copiado sin explicación.
- [x] `gdb.md` demuestra la validación dinámica.
- [x] Creé tag `lab-reverse-v1`.

🌐 Módulo opcional — 20–30 min con Burp Suite

Burp no reemplaza a Ghidra: Burp analiza tráfico HTTP; Ghidra/GDB analizan el binario. Para incorporar Burp sin depender de publicar servicios, usa exclusivamente los laboratorios deliberadamente vulnerables de PortSwigger Web Security Academy.

Ruta sugerida: abre el tutorial oficial, mira los videos de introducción y practica únicamente **Proxy** → **Intercept** → **Repeater** sobre el laboratorio asignado.

1. Inicia Burp Community.
2. Usa el navegador integrado de Burp.
3. Abre únicamente el laboratorio de Academy asignado.
4. Intercepta una petición.
5. Envíala a Repeater.
6. Cambia un parámetro inocuo y observa request/response.
7. Captura evidencia y explica qué diferencia hay entre análisis web y reverse engineering.

🧠 Preguntas de análisis

4. ¿Qué ventaja tuvo Ghidra sobre objdump?
5. ¿Qué confirmó GDB que el análisis estático por sí solo no demostraba?
6. ¿Por qué Burp Suite no es una herramienta de ingeniería inversa de binarios?
7. ¿Qué controles de desarrollo evitarían embebidos inseguros de secretos en software real?

🏆 Puntuación

95 / 95 puntos base

El módulo Burp es bono/actividad complementaria y no afecta el progreso automático del crackme.

🎤 Cierre de 3 minutos por equipo

Sin mostrar únicamente pantallazos. El equipo debe poder explicar:

1. **Qué observó** inicialmente.
2. **Qué hipótesis** formuló.
3. **Qué función o condición** encontró.
4. **Cómo la confirmó** en ejecución.
5. **Qué enseñanza de desarrollo seguro** extrae del crackme.

Uso académico autorizado · FDSI · Los retos incluidos no contienen malware, persistencia ni funcionalidad destructiva.